Date: 22nd April 2026 **Project Supervisor:** Prof. Bhaskar Biswas

Phase II Report: Swarm Intelligence Metaheuristics for Influence Maximization in Social Networks

Rahul Kumar Das

Roll No. 24075102

Department of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi

Exploratory Research Project | Semester IV

Contents

1 Abstract

Building upon the bio-inspired foundations of Phase II, this report transitions from Genetic Algorithms to the implementation of Swarm Intelligence metaheuristics for the Influence Maximization (IM) problem. To overcome the exponential time complexity of traditional Monte Carlo simulations, a Local Influence Heuristic (Expected Diffusion Value, $EDV_{[2]}$) was engineered, enabling scalability to massive networks. Two distinct swarm models were benchmarked: the constructive Ant Colony Optimization (ACO-IM) and the navigational Discrete Particle Swarm Optimization (DPSO). Results across varied topologies, including a 36,000-node peer-to-peer network, demonstrate that while ACO-IM yields a superior influence spread via localized heuristic searches, DPSO offers drastically faster execution times through vectorized continuous-to-discrete spatial transformations.

2 Introduction

Information diffusion models capture the propagation of behaviors or ideas across social graphs. Finding the optimal starting nodes (seed set) to maximize this spread is a well-documented NP-hard problem. While Phase II proved that metaheuristics like Genetic Algorithms could approximate the mathematical optimal, stochastic crossover operations lack the "collective memory" utilized by swarm-based algorithms.

Swarm Intelligence mimics decentralized, self-organized systems. This report benchmarks two state-of-the-art approaches: Ant Colony Optimization, which constructs solutions piece-by-piece using pheromone trails, and Particle Swarm Optimization, which navigates a continuous mathematical space to isolate discrete network nodes.

3 Problem Formulation

Let the social network be represented as a directed graph $G = (V, E)$, where V is the set of nodes and E is the set of edges. Given a diffusion model and a budget k, the objective is to find a seed set $S \subset V$ such that $|S| = k$, which maximizes the expected number of active nodes, $\sigma(S)$.

The optimization problem is defined as:

$$S^* = \arg \max_{S \subseteq V, |S|=k} \sigma(S) \tag{1}$$

4 Network Datasets

To rigorously evaluate the scalability and robustness of the metaheuristic model, experiments were conducted across three distinct network topologies from the Stanford Network Analysis Project (SNAP). These datasets provide a necessary mix of edge densities, directed versus undirected influence flows, and varying degrees of community clustering.

- **ca-GrQc (Collaboration Network):** A directed graph representing scientific collaborations in General Relativity and Quantum Cosmology.

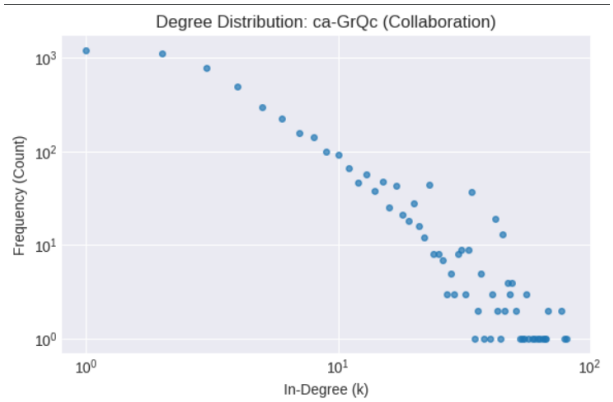

Figure 1: Degree distribution of the ca-GrQc dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 5242, 28980 \rangle$. As a sparse network, it serves as the primary baseline environment for establishing the mathematical ceiling against the Greedy algorithm.

- **ego-Facebook:** An undirected graph representing dense social circles and community clusters.

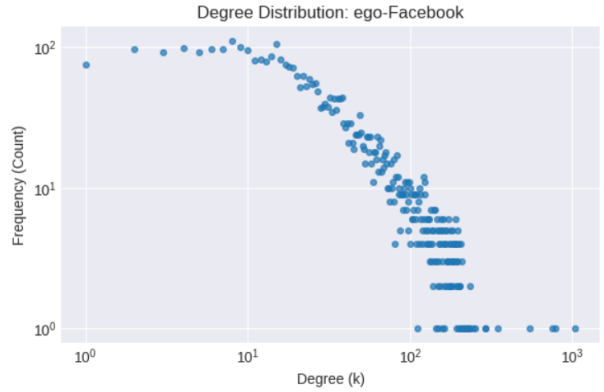

Figure 2: Degree distribution of the ego-Facebook dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 4039, 88234 \rangle$. The high edge density provides a stress test for algorithmic susceptibility to overlapping influence zones (local optima).

- **p2p-Gnutella30:** A massive directed peer-to-peer file-sharing network.

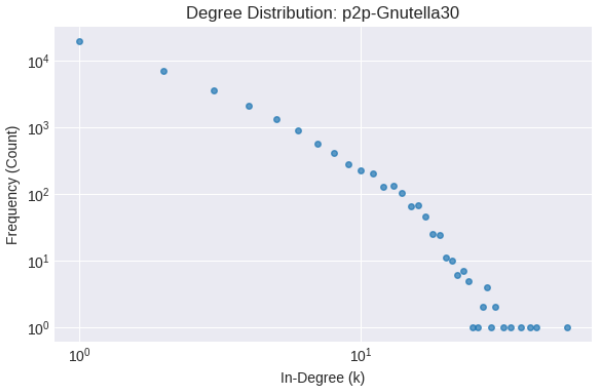

Figure 3: Degree distribution of the p2p-Gnutella30 dataset plotted on a log-log scale. The network consists of $(\langle V, E\rangle = \langle 36682, 88328\rangle)$. This dataset acts as the ultimate scalability benchmark, possessing a node volume that renders traditional Greedy algorithms intractable.

5 Methodology

5.1 The Local Influence Heuristic ($EDV_{[2]}$)

To circumvent the immense computational overhead of executing thousands of Monte Carlo simulations per generation, an Expected Diffusion Value within a two-hop area ($EDV_{[2]}$) was implemented as the primary fitness evaluator. This mathematical formulation accurately estimates spread by calculating the probability of 1-hop neighbor activation, multiplied by a 2-hop expansion factor representing "friends of friends".

```python
def edv_2_fitness(G, S, p=0.1):
    S_set = set(S)
    k = len(S_set)
    # Identify 1-Hop and 2-Hop neighbors...

    sigma_1 = 0
    for u in S_set:
        prob_failure = 1.0
        for v in G.successors(u):
            if v in N1:
                prob_failure *= (1 - p)
        sigma_1 += (1 - prob_failure)

    two_hop_sum = 0
    for w in N2:
        in_edges = sum(1 for v in G.predecessors(w) if v in N1)
        p_A_w = 1 - ((1 - p) ** in_edges)
        two_hop_sum += (p_A_w * in_edges)

    expansion = 1 + (1 / len(N1)) * two_hop_sum
```

```
    return k + (expansion * sigma_1)
```

Listing 1: Expected Diffusion Value ($EDV_{[2]}$) Evaluator

5.2 Ant Colony Optimization (ACO-IM)

The ACO-IM architecture employs a constructive search strategy. Artificial ants build a seed set of size k node-by-node. Node selection is governed probabilistically by a combination of historical pheromone levels (τ) and localized heuristic quality ($\eta = \rho \cdot \tau$). To prevent premature convergence (stagnation), an exploration decay probability $p_I = \frac{\log(I)}{\log(I_{max})}$ forces ants to execute random node selections during early iterations.

```
def run_aco_im(G, k, pop_size=15, I_max=10, rho=0.2, alpha=0.5, beta
    =0.5):
    all_nodes = list(G.nodes())
    tau = {node: 1.0 for node in all_nodes}
    # ...
    while len(current_seed) < k:
        if random.random() >= p_I:
            selected = random.sample(list(avail), 1)[0]
        else:
            weights = []
            for v in list(avail):
                eta_v = rho * tau[v]
                weights.append((tau[v] ** alpha) * (eta_v ** beta))
            # Select node probabilistically based on weights
    # ...
```

Listing 2: ACO-IM Constructive Search and Pheromone Update

5.3 Discrete Particle Swarm Optimization (DPSO)

Unlike the constructive approach of ants, DPSO utilizes a navigational search strategy. Particles possess a continuous multi-dimensional velocity vector. Because network nodes are discrete binary choices (selected vs. not selected), a mathematical bridge is required. The algorithm executes a Sigmoid Transformation to squash velocities into probabilities, followed by a Top-k filter to strictly enforce the budget constraint.

```
def run_discrete_pso(G, k, pop_size=15, I_max=10, w=0.7, c1=1.5, c2=1.5)
    :
    # ...
    # PSO Velocity Update Formula
    cognitive_pull = c1 * r1 * (pbest_pos[i] - positions[i])
    social_pull = c2 * r2 * (gbest_pos - positions[i])
    velocities[i] = (w * velocities[i]) + cognitive_pull + social_pull

```

```
    # Sigmoid Transformation Bridge
    clipped_vel = np.clip(velocities[i], -10, 10)
    probs = 1 / (1 + np.exp(-clipped_vel))

    # Top-k Filter to snap back to discrete graph choices
    top_k_indices = np.argsort(probs)[-k:]
    positions[i] = np.zeros(N)
    positions[i, top_k_indices] = 1
    # ...
```

Listing 3: DPSO Velocity Update and Sigmoid Transformation

6 Experimental Setup

To ensure a rigorous comparison, parameters were standardized across both Swarm architectures.

- **Swarm Size (P):** 15
- **Generations/Iterations (I_{max}):** 10
- **ACO Parameters:** $\rho = 0.2$, $\alpha = 0.5$, $\beta = 0.5$
- **DPSO Parameters:** $w = 0.7$, $c_1 = 1.5$, $c_2 = 1.5$

7 Results and Discussion

7.1 Influence Spread Accuracy

Benchmarking the constructive ACO against the navigational DPSO revealed that ACO consistently achieved a marginally higher Expected Diffusion Value ($EDV_{[2]}$) across all three datasets (Figure 4). This indicates that ACO's localized heuristic integration (η) during the step-by-step construction phase allows for a more refined search space exploration compared to DPSO's post-flight blanket evaluations.

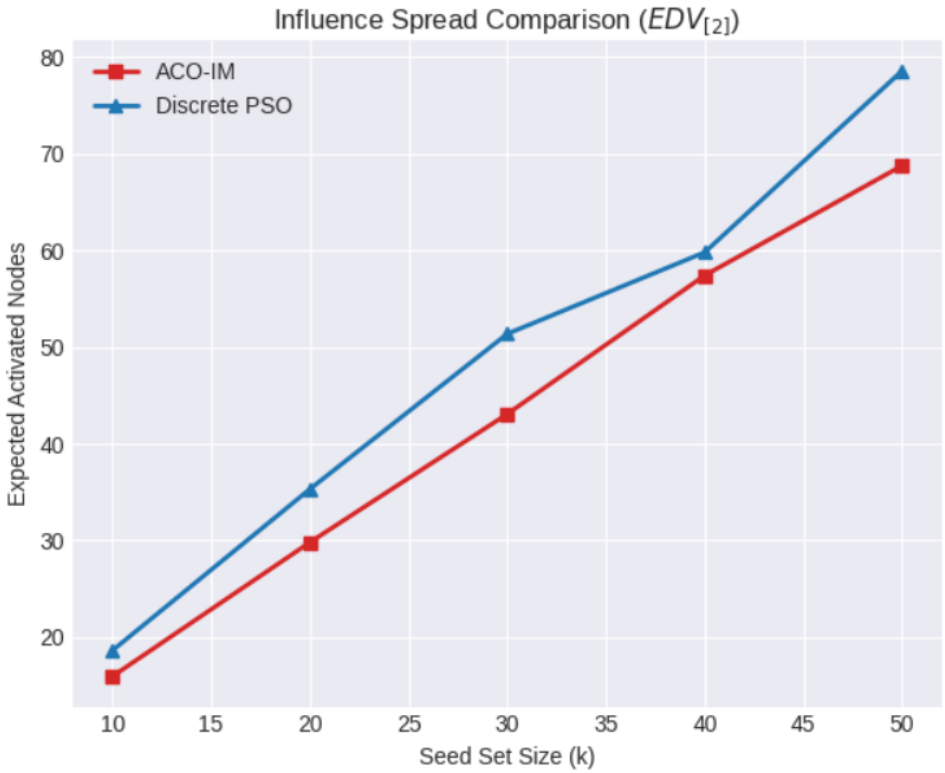

Figure 4: Influence Spread Comparison ($EDV_{[2]}$) across three network topologies.

7.2 Computational Scalability

While ACO proved superior in output quality, DPSO demonstrated a decisive advantage in computational efficiency (Figure 5). Because the DPSO architecture leverages vectorized continuous mathematics to update multi-dimensional particle arrays simultaneously, it bypassed the algorithmic bottleneck of ACO's sequential probability evaluations. On the 36,000-node *p2p-Gnutella30* dataset, DPSO executed significantly faster than ACO, showcasing robust scalability for massive graphs.

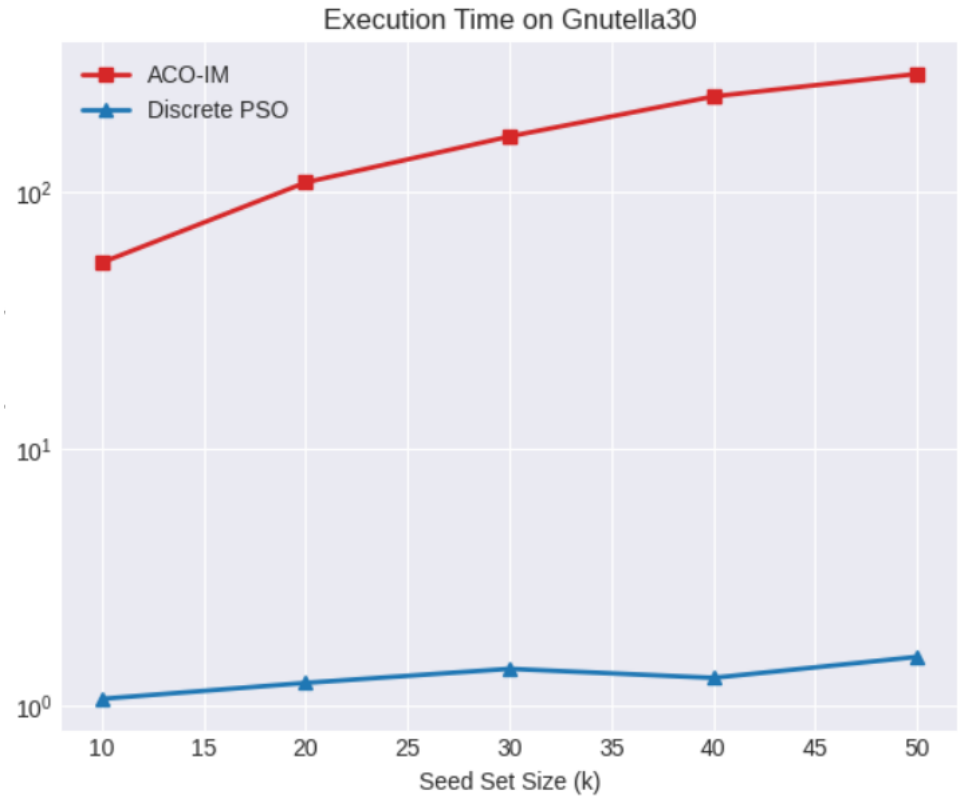

Figure 5: Execution Time Comparison evaluated on a logarithmic scale.

7.3 Algorithmic Convergence

Internal validation of both swarms confirms distinct learning curves (Figure 6). DPSO experiences sharp vertical improvements early in execution as the "Global Best" vector rapidly pulls the entire swarm toward a concentrated sector of the search space. Conversely, ACO demonstrates a smoother, steadier climb, as the pheromone trails gradually solidify and the exploration decay (p_I) seamlessly transitions the swarm into exploitation mode.

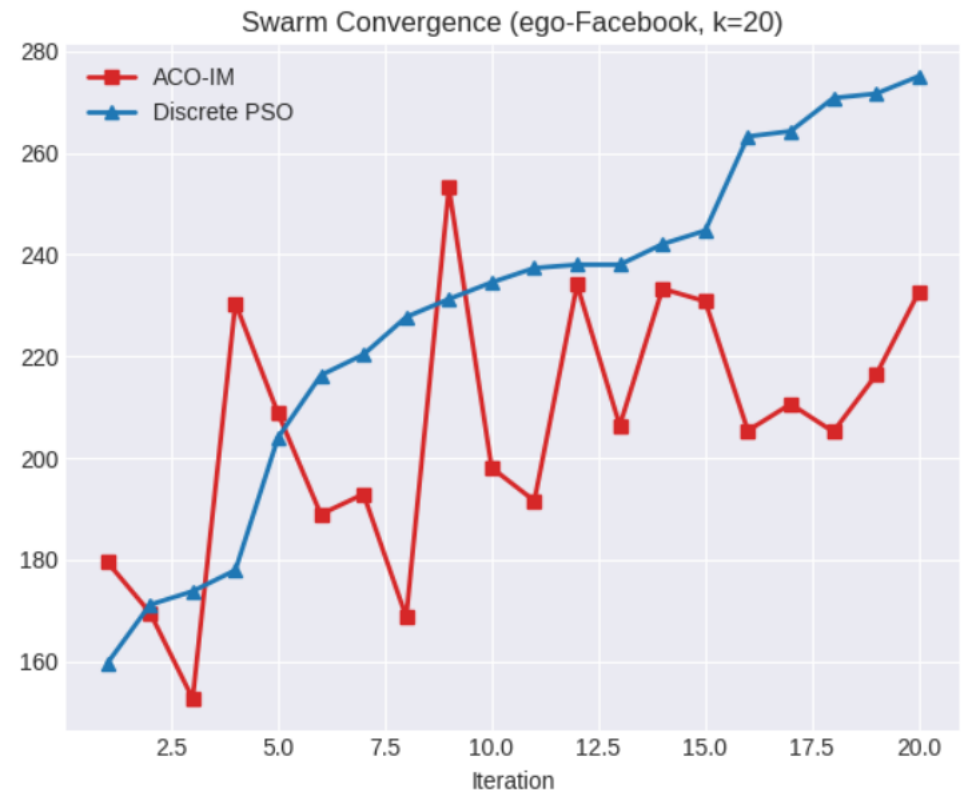

Figure 6: Swarm Convergence: Best Fitness Score over Iterations (evaluated on $k = 30$).

8 Conclusion

This phase of the project confirms that Swarm Intelligence provides a highly potent framework for the Influence Maximization problem. By abstracting the simulation overhead into the $EDV_{[2]}$ heuristic, both algorithms easily scaled to massive networks. The results underscore a fundamental engineering trade-off: Ant Colony Optimization (ACO) is preferable when absolute maximal influence spread is paramount, while Discrete Particle Swarm Optimization (DPSO) is superior when execution time and scalability are the primary constraints.

Name : Rahul Kumar Das
Roll No. : 24075102
Dept. : CSE(BTech)